

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ И ВЕБ-
ИНТЕРФЕЙСА

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков проектирования приложения интернета вещей и интерфейса для отображения состояния системы.

2 Задание

1. В рамках предложенной (или своей) темы спроектируйте систему, состоящую из 4-5 уникальных классов. Подумайте имеют ли классы общие методы и свойства, если таковые есть, выделите их в абстрактный класс. Обозначьте связь элементов системы (Подсказка: не лишним будет создать отдельный объект, который не имеет физического воплощения, а отвечает только за обработку и передачу данных между другими объектами, если это необходимо, в моём примере – это MainControlUnit). Подумайте какие данные в системе будет хранить база данных, обозначьте её отдельной сущностью, связями обозначьте какие данные и с каких вещей она должна хранить. Поскольку следующие лабораторные подразумевают написание кода на Python, в котором модификаторы доступа носят рекомендательный характер, ими можно пренебречь.

2. В рамках предложенной (или своей) темы спроектируйте, как минимум два интерфейса. Два интерфейса могут быть нужны, например, для пользователя и администратора. На примере умного дома это может быть интерфейс ребёнка, который только отображает некоторые данные и позволяет управлять некоторыми из вещей и интерфейс родителя, которые может позволять более глубоко настраивать управление домом. Подумайте какие данные лучше скрыть от одного пользователя, какие лучше показать, как лучше их отобразить.

Вариант задания - Умная теплица.

3 Проектирование слоя цифровых двойников

Была разработана диаграмма классов в нотации UML. Основой системы является абстрактный класс Device, который определяет общие свойства и поведение для всех физических устройств умной теплицы. От него наследуются классы Sensor (датчик) и Actuator (исполнительное устройство). Управление взаимодействием между устройствами и обработка данных возложены на класс MainControlUnit, который выступает в роли центрального контроллера. Для долговременного хранения информации о показаниях датчиков и событиях системы введён класс Database. UML-диаграмма представлена на рисунке 1.

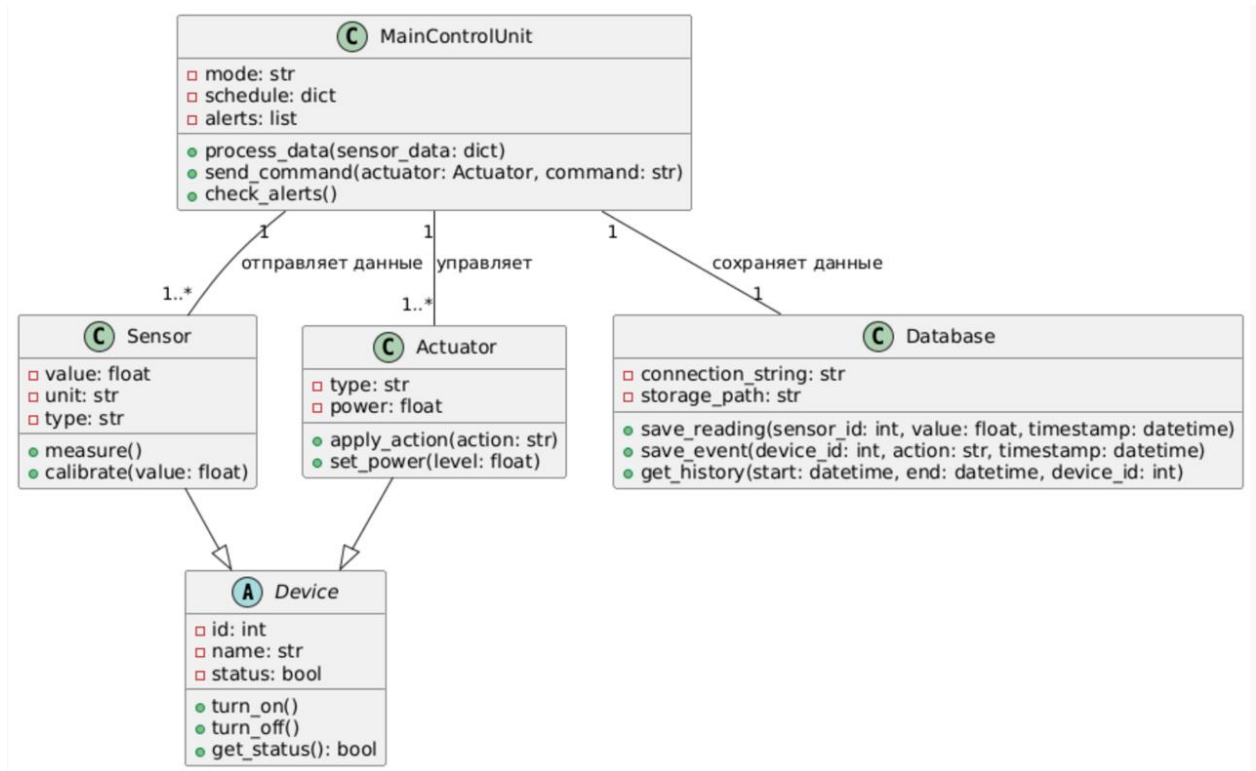


Рисунок 1 – UML-диаграмма

Класс Device является абстрактным и содержит общие атрибуты: уникальный идентификатор id типа int, название устройства name типа str и флаг состояния status типа bool. Методы класса обеспечивают базовое управление: turn_on() включает устройство, turn_off() выключает его, а get_status() возвращает текущее состояние.

Класс `Sensor` наследует все свойства класса `Device` и дополняет их специфическими атрибутами: текущее измеренное значение `value` (float), единица измерения `unit` (str), тип датчика `type` (str). Метод `measure()` имитирует процесс измерения и обновляет значение, а `calibrate(value)` позволяет откалибровать датчик, передавая эталонное значение.

Класс `Actuator` также наследуется от `Device` и описывает исполнительные устройства, такие как нагреватель, вентилятор или лампа. Его атрибуты: тип устройства `type` (str) и мощность `power` (float). Метод `apply_action(action)` выполняет переданную команду (например, полив, включение обогрева), а `set_power(level)` изменяет мощность работы устройства.

Класс `MainControlUnit` представляет центральный контроллер, который не имеет физического двойника, но реализует логику управления. Его атрибуты: режим работы `mode` (str) со значениями “auto” или “manual”, расписание автоматических операций `schedule` (dict) и список активных тревог `alerts` (list). Метод `process_data(sensor_data)` анализирует поступившие с датчиков данные и при необходимости инициирует управляющие воздействия. Метод `send_command( actuator, command)` отправляет команду конкретному исполнительному устройству. Метод `check_alerts()` проверяет наличие критических значений и формирует уведомления.

Класс `Database` предназначен для хранения данных. Атрибуты: строка подключения `connection_string` (str) и путь к хранилищу `storage_path` (str). Метод `save_reading(sensor_id, value, timestamp)` сохраняет показание датчика с привязкой ко времени. Метод `save_event(device_id, action, timestamp)` фиксирует событие, связанное с работой устройства. Метод `get_history(start, end, device_id)` возвращает историю измерений или событий за указанный период, с возможностью фильтрации.

4 Проектирование пользовательских интерфейсов

Для системы “Умная теплица” целесообразно выделить две роли пользователей: агроном (оператор) и инженер (администратор). Каждая роль имеет свои задачи и, соответственно, разный набор отображаемых данных и доступных функций.

Интерфейс агронома (рисунок 2) предназначен для повседневного мониторинга состояния теплицы и быстрого реагирования на нештатные ситуации. Основное требование - наглядность и простота. На главном экране размещается схематичное изображение теплицы, разбитое на секции. Для каждой секции крупным шрифтом выводятся текущие значения температуры, влажности и освещённости, а также цветовая индикация отклонений.

В отдельном блоке отображаются активные тревоги с возможностью немедленного выполнения типового действия, например, включения вентиляции в проблемной зоне. Под схемой располагается упрощённый график изменения температуры за последние сутки, позволяющий быстро оценить тренд. В нижней части экрана находятся крупные кнопки для часто используемых операций: “Полить все”, “Вентиляция”, “Свет”. Такой интерфейс позволяет агроному без лишних действий оценить обстановку и принять меры.



Рисунок 2 – Интерфейс агронома (оператора)

Интерфейс инженера (рисунок 3) предоставляет более детальную информацию и расширенные возможности настройки. Левая панель содержит навигационное меню с разделами: датчики, актуаторы, логи, настройки, права доступа. В центральной части отображается таблица всех датчиков с указанием идентификатора, типа, текущего значения, статуса и кнопкой для калибровки. Рядом может быть размещён детальный график, на котором пользователь может выбирать отображаемый параметр и временной период. Ниже располагается лог событий, где в хронологическом порядке фиксируются все значимые действия системы (включение/выключение устройств, срабатывание тревог, изменение режимов). Особое внимание

уделено блоку настройки автоматизации: здесь администратор может создавать новые задания, редактировать существующие и удалять их. Также предусмотрен раздел управления пользователями, где назначаются роли и права доступа.



Рисунок 3 – Интерфейс инженера (администратора)

Оба интерфейса выполнены в виде схематических макетов, в которых обозначены основные функциональные зоны. При проектировании учитывались принципы информационной безопасности: агроном не видит административные настройки и не имеет доступа к редактированию правил автоматики, в то время как инженер видит всю информацию и может её изменять.

Вывод

В ходе выполнения лабораторной работы была спроектирована система интернета вещей для умной теплицы. Разработана диаграмма классов слоя цифровых двойников, включающая абстрактный класс Device, классы Sensor, Actuator, MainControlUnit и Database. Определены отношения наследования и ассоциации с указанием кратности, что позволяет в дальнейшем реализовать программную модель на языке Python.

Созданы эскизы двух пользовательских интерфейсов - для агронома и для инженера, различающиеся по составу отображаемой информации и функциональности. Интерфейс агронома ориентирован на оперативный контроль и простоту, интерфейс инженера - на детальный анализ и настройку системы.

Таким образом, цель работы достигнута: приобретены навыки проектирования приложений интернета вещей, включая построение диаграмм классов и разработку интерфейсов с учётом ролей пользователей.